Communication Protocol

General Description

Communication between hostsystem and smart-battery-pack is enabled by using a serial interface in UART mode (V.24 protocol). The hostsystem acts as a master that starts communication and requests data from the smart-battery-pack, that receives data executes all related actions and returns a response containing an acknowledge or not acknowledge and data. All data are transmitted in packets, that are different to master and slave.
For serial communication with the smart-battery-pack you must use following parameters: 19200 baud, 8 databit, no paritycheck, 1 stopbit (19200, 8, N, 1).

Detailed Description

Known commands and packets are shown below.
All commands are case sensitive and lower case only.

1. Read status

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Read status	Master → Slave	?	s	0	0	chs[1]	Reading statusbyte, Alarm and Warning
	Slave → Master	!	s	awsuTtHc[2]	nhrdmvig[2]	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))
[2] from MSB to LSB,

a = Alarm	n = NTC defect
w = Warning	h = temperature not in valid region (hot, cold)
s = charging is via serial command enabled	r = V_{bat} is short (< 0,6V)
u = -dU criteria	d = V_{bat} is deep (< 3.1V)
T = dT/dt criteria	m = V_{bat} is minimal (< 3.8V)
t = time criteria	v = V_{bat} is over limit (>6,57V)
H = hallsensorstatus 1 = active, 0 = inactive	i = discharge current is too high
c = accupack is charging (don't activate accutest)	g = charge current is too high

2. Read capacity

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Read capacity	Master → Slave	?	c	0/0xA5[1]	0	chs[2]	Reading capacity, integer-value, lowbyte first (in mAh)
	Slave → Master	!	c	low-byte	hi-byte	chs	

[1] 0x00 = current capacity, 0xA5 = last full charged capacity.
[2] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))

3. Read version

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Read version	Master → Slave	?	v	0	0	chs[1]	Reading software version
	Slave → Master	!	v	version[2]	Tbnnnnnn[3]	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))
[2] value / 100 = versionnumber, ex. 100 = SW 1.00
[3] from MSB to LSB, T = 0 (NiMH), T = 1 (LiIon), b = beta-Version, nnnnnn = nominal capacity in 100mAh, Range from 100mAh to 6400mAh

4. Set to dischargemode

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Set to discharge-mode	Master → Slave	!	d	0/0xA5[1]	0	chs[2]	Inhibit battery-pack self-control, charging disabled
	Slave → Master	!	d	caxxxxxx[3]	0	chs	

[1] 0 = daycounter reset at 30min Rem.Time, 0xA5 = daycounter reset at BatteryEmptyPoint
[2] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))
[3] from MSB to LSB, c = chargemode active, a = 1: AccuMaintenance running, a = 0: AccuMaintenance finished, x = reserved, this mode is automatically left after 15s.

5. Read daycounter (Tageszähler)

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Read daycounter (Tages-zähler)	Master → Slave	?	t	0	0	chs[1]	Reading daycounter
	Slave → Master	!	t	low-byte	0	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))

6. Read remaining time

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Read remaining time	Master → Slave	?	r	0	0	chs[1]	Reading remaining time in minutes
	Slave → Master	!	r	low-byte	hi-byte	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))

7. Set remaining time

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Set remaining time	Master → Slave	!	r	low-byte	hi-byte	chs[1]	Reading remaining time in minutes
	Slave → Master[2]	!	r	low-byte[2]	hi-byte[2]	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))
[2] In Byte3 and Byte4 the new remaining time in minutes is returned. If this integer is not equal to the sent integer, the remaining time is not set.

8. Get accutest result

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Start accutest	Master → Slave	!	a	0	0	chs[1]	Starting the accutest
	Slave → Master	!	a	iuxxxxep[2]	xxxxxxxx[2]	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))
[2] from MSB to LSB, i = current lower than 200mA, u = V_{bat} lower than 4.6V, e = accutest is enabled, p = accutest is passed, x = reserved.

9. Get serialnumber low word

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Any other "read" sequence	Master → Slave	?	1	0	0	chs[1]	Reading serialnumber low word
	Slave → Master	!	1	SN-Byte0	SN-Byte1	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))

10. Get serialnumber high word

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Any other "read" sequence	Master → Slave	?	2	0	0	chs[1]	Reading serialnumber high word
	Slave → Master	!	2	SN-Byte2	SN-Byte3	chs	

[1] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))

11. Any other "read" sequence

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Any other "read" sequence	Master → Slave	?	byte[1]	0	0	chs[2]	wrong command was sent. No answer from module.
	Slave → Master						

[1] „byte" is not s, c, v, t, r, 1, 2
[2] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))

12. Any other "set" sequence

Instruction		Instructionformat					Operation
		Byte1	Byte2	Byte3	Byte4	Byte5	
Any other "set" sequence	Master → Slave	!	byte[1]	0	0	chs[2]	wrong command was sent. No answer from module.
	Slave → Master						

[1] „byte" is not d, a, r
[2] chs = checksum (= 0 – (Byte1+Byte2+Byte3+Byte4))